Feature: Calendars

From build-plan: feature 12b

Size: heavy - three separate risks with real logic: reading a week for one person or place from four sources (hours, closed days, held time, Google), a booking made by hand that must be a real booking, and time off over bookings that moves or cancels them in the same save.

Branch: feature/12b-calendars

Status: verified 2026-10-12: 12b.1, 12b.2 and 12b.3 built and reviewed, every step finding closed (F-412 to F-427); shared 188 and backend 1046 tests pass, the frontend builds and lints. Then the final review of the whole feature.

Goal

Every person and every place has a calendar in the app. The owner sees a week of it: the bookable hours, closed days, bookings with their customer, time off, and the person's Google busy times. He adds things by hand: a phone estimate or a walk-in as a real booking (the customer, the lead, the texts), and time off or a blocked afternoon. He can place work any time someone is not off and not already busy; bookable hours only limit customers. Marking someone off over existing bookings moves or cancels each of them in the same save. Until this, nothing can be put in by hand, and the website could sell a slot already given away on the phone.

In scope

- **The Calendar page**, `/calendar`, in the dashboard's main list after Leads: pick a person or a place, see one week (Monday to Sunday, the business's time zone), previous and next week.
- **Each day shows**: the bookable hours (12a; none on a closed day, 12e, marked closed); every active booking (time, service, customer's name, a link to the lead); time off; for a person with Google connected, the Google busy blocks of their ticked calendars (12h) apart from the app's own, or "Google could not be read" for that week, the rest still shown.
- **Book by hand** from the page: a service the person does, the person, a start (any time nobody it needs is busy, inside or outside bookable hours, never an earlier day), the customer's name, and if the owner has them their phone, email and the address when the service asks one, and a note (amended at 12b.2's review, F-417: the owner is held to no contact rule, 12f). The same booking path the website uses (`bookTime` , source `manual`): a lead in the first stage, the timeline entry, the Google event, the confirmation and texts as the business has set them (6, 8b, 12g).
- **Time off** for a person or a place: a start and an end, any length (an afternoon, a day, a week). Saved as `commitment` rows of kind `time_off` ; removing one releases it. When it covers active bookings of that person or place, the save must carry a choice for each: **cancel** (the customer told, as the owner's

cancel since 12e) or **move** (a new start, and the same person or another who does the service; the customer told, the Google event following, as 7b's move). The whole save is one transaction: the time off, every move and every cancel, or nothing (Decisions, 3).
- **Who sees what** (Decisions, 2): a role that may change the business sees every calendar and adds everything; any other member sees only their own person's calendar, read-only.

Out of scope

- Moving or cancelling a booking from the calendar outside time off (the lead page and 12e cover cancel; a general owner move is a later item).
- Drag and drop, a day view of everyone side by side, and the final look (logic first, layout later).
- Recurring time off, and a reason or label on time off.
- Time off written into Google; other providers.

Build loop

Approved one step at a time (AGENTS.md). Heavy: `workflow.stepReview` is `every`, `checkpointCommits` enabled. After a step's plan has Frank's yes, nothing stops until the review: build, tests and checks, tick the box, write the build log entry and commit it to buildlogs, commit and push the step to `feature/12b-calendars`, `/audit` scoped to the step, then the independent review. The planned stop is after the review. Blocking findings (P0/P1) are fixed by default; P2/P3 are recorded and carried. `/complete` makes the final pass and merges on Frank's yes.

Build steps

- ☐ **12b.1 See each calendar.** "Summit's owner opens Calendar, picks Carlos and sees this week: his hours Monday to Friday, Thursday closed for a holiday, Tuesday's 10:00 booking with the customer's name, Friday afternoon off, and the busy block from his Google. Picking the van shows the bookings that use it. Next week shows next week. A member who is only Carlos sees Carlos's calendar and nobody else's."
  - Backend: `GET /schedule?resourceId=&weekOf=YYYY-MM-DD` (the business from the session): `{ canEdit, timezone, resources, resource, days: [{ date, closed, hours, items }], google }`, items `{ kind: "booking" | "time_off" | "google_busy", startsAt, endsAt, booking? }`. Reuses `resolveBookableHours`, `closed-days-between.ts`, `findCommitments` and `getBusyTimes`.
  - Frontend: `app/(dashboard)/calendar/page.tsx`, "Calendar" in the dashboard list, the week screen. **Done when:** the route tests pass, and by hand on Summit the scene above shows.
- ☐ **12b.2 Book by hand.** "On Carlos's Thursday the owner books Mrs Lee's phone estimate at 4:30 pm, after Carlos's hours: it shows on the calendar, Mrs Lee is a new lead, her confirmation goes as

Summit's settings say, and the event is in Carlos's Google. 4:30 on a day Carlos is off is refused, and so is a day already gone."

  - Backend: `POST /schedule/bookings` calling `bookTime` with source `manual` and the owner's login.
  - Frontend: the booking form on the calendar. **Done when:** the route tests pass, and by hand on Summit the scene above works.

- [x] **12b.3 Time off.** "The owner marks Carlos off all Friday. Carlos has two bookings Friday: the page lists them in the same form; the owner moves one to Monday 9:00 with Carlos and cancels the other, then saves once. Friday is off, one customer is told of the new time and one of the cancel. If Monday 9:00 was taken meanwhile, nothing is saved, the cancel included. Removing the time off opens Friday again."
  - Backend: `POST /schedule/time-off` and `DELETE /schedule/time-off/:id`; an owner move (any time nobody is busy, its own old time not in the way, the person one who does the service); the save in one transaction with the cancel and move internals (check in the build that both can run inside a caller's transaction; give them that if not).
  - Frontend: the time off form with its list of bookings in the way. **Done when:** the route tests pass, the four Simulate cases are saved tests under their names, and by hand on Summit the scene above works.

Files / areas

- `frontend/app/(dashboard)/calendar/`, `frontend/components/schedule/` (the week, the forms), the dashboard list in `frontend/components/dashboard/dashboard-shell.tsx`, `frontend/lib/api-client/schedule/`
- `backend/routes/schedule-routes.ts` and its test, mounted at `/schedule` in `backend/app.ts` under the dashboard CORS, CSRF and no-store middleware; `backend/lib/schedule/` (the week, the time off save, the owner move); `backend/lib/booking/` (`bookTime`, cancel and move internals)
- `packages/shared/zod-validation/` (the three bodies); no migration: `commitment` holds time off since 5a

Data / contracts

All under the dashboard middleware and the `booking` module. The business always comes from the session; every query filters on it first. Reads: a role with `organization: ["update"]` any resource of the business, anyone else only the person linked to their login (`403` otherwise). Writes need `organization: ["update"]`.

`GET /schedule?resourceId=&weekOf=` `weekOf` a local date, the week its Monday starts. `resourceId` optional: missing, the caller's own person, or the first person for an owner without one. `404` for a

resource not of the business. `google: "read" | "not_connected" | "unreadable"`. A booking item carries `{ id, leadId, customerName, serviceName, personName }`; text rendered by React as text.

`POST /schedule/bookings` `{ bookingLinkId, personId, date, startMinute, customer: { name, email, phone }, location, details }`, strict (amended at 12b.2: the business's own clock, a date and a minute of the day, turned into a moment in its zone by the API, so the page needs no time-zone sums; an hour skipped in spring is `400` on `startMinute`). `201 { booking }`. `409 time_taken` when anyone it needs is busy, `400 in_the_past` for an earlier day, `400` on a field the schema refuses; never a missing contact or address, which the owner may leave for later (F-417). `409 reconnect_needed` when the person's Google needs reconnecting, so their time cannot be checked (F-419); `503 unavailable` when Google does not answer.

`POST /schedule/time-off` `{ resourceId, from: { date, minute }, to: { date, minute }, bookings: ({ bookingId, action: "cancel" } | { bookingId, action: "move", date, startMinute, personId })[] }`, strict, end after start (amended at 12b.3: the business's own clock, as 12b.2, `to.minute` up to 1440 for the end of a day). `201 { timeOff }`. Amended after the 12b.3 review (F-421 to F-426) to the answers the route gives: `409 no_business_hours` when the business has no hours yet; `404 not_found` for another business's person or place; `409 already_off` when time off is already in the span ("Already off from … to …: remove it first"), said before any booking is chosen for; `409 bookings_in_the_way { bookings }` when an active booking in the span has no choice, a choice names one not in the way, or one has already begun; per move, each with `{ bookingId }`: `400 bad_request` for a person who does not do the service, `400 in_the_past` for a time already gone, `409 time_taken` for a move into the time off itself or to a time not free, `503 unavailable` when Google does not answer, `409 bookings_in_the_way` (no `bookings`) when that booking was cancelled, begun or left meanwhile; `400 { field }` for a field the schema refuses or a clock time skipped in spring, `bookings.N.startMinute` for a move's; `409 time_taken` with no `bookingId` when a move's time or the span itself is taken, or a booking in the way changed, meanwhile. For a place, a move leaves that place over the time off: the same start in another room is a move, never "already there". Nothing saved on any refusal.

`DELETE /schedule/time-off/:id` `204`; `404` for another business's or a booking's row.

Testing

`npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend` (local Postgres, migrated and seeded), `npm run build` and `npm run lint` for the frontend. One test per rule that would hurt if broken:

- 12b.1: the week shows hours, closed days, bookings, time off and Google busy apart, in the business's zone; Google unreadable still shows the rest; a member sees only their own calendar; another business's resource is 404.
- 12b.2: a booking by hand outside bookable hours is made, with a lead and its jobs; a busy time, time off and a past time are refused; a member without the permission cannot book.

- 12b.3, the Simulate cases under their names: "time off with no bookings in the way is saved"; "time off over a booking with no choice is refused, and nothing is saved"; "one booking moved and one cancelled in one save: the time off is saved and both customers are told"; "a move to a time taken meanwhile saves nothing, not even the cancel". Also: removing time off frees the time.

Decisions (made while planning, Frank's to overrule)

1. **One person or place at a time, a week at a time.** One Google read per view; everyone side by side is a later look. 2. **Owners see everyone; a member sees only themselves, read-only.** Customer names stay with the roles that run the business. 3. **Time off over bookings is all or nothing.** A move that fails leaves nothing half done: no time off saved over a booking still standing, no customer told of a cancel that did not finish. 4. **Time off and a blocked afternoon are the same thing**: a span, any length. No reason field.

Notes for the AI

- `bookTime` with source `manual` already skips the form's contact and answer rules and books any time nobody is busy (`findBookingChoices` `byOwner`); 12b.2 is mostly the route and the form.
- `findBookingChoices` refuses `movingBooking` with `byOwner` today; 12b.3 adds the owner move.
- Google is read, never written, for time off. No outside settings change in any step.

Implementation walkthrough

Feature 12b was specced heavy, three steps, one per real risk, and kept them: 12b.1 in `3aafcf3` (fixes `24a4a37` , `0ed3baa`), 12b.2 in `9b12f37` (fixes `118065a`), 12b.3 in `5aed73b` (fixes `107e315` , `dda246d`), the checkpoint for the final review in `b2d83d4` . Every step had a fresh reviewer and a check of its fixes until nothing new was found: F-412 to F-427, sixteen findings, all closed. The final review of the whole feature passed with two P3s left open and carried (F-428, F-429). No migration: `commitment` has held time off since 5a.

the week (`backend/lib/schedule/` , `routes/schedule-routes.ts`)

`GET /schedule?resourceId=&weekOf=` , mounted at `/schedule` in `app.ts` under the dashboard CORS, CSRF and no-store middleware, the business always from the session. `find-schedule-week.ts` reads what the booking window already reads, and decides nothing about who is free: the hours from `resolveBookableHours` (now taking `coverDays` , so the owner sees a whole week however near the customers' horizon ends), a day's windows through the new `windows-on-date.ts` (shared with the free-times rules, so a one-off date replaces the week in one place), closed days, standby days for a place (no bookable hours shown there, F-413), the held time from `findCommitments` , and a person's Google busy blocks apart, or `unreadable` for the week with the rest still shown. `monday-of.ts` gives the week's Monday in the business's zone. An item across several local days shows on each, and a week with the

spring clock change has seven days (F-415). A role that may change the business reads any calendar; anyone else only the person linked to their login.

Google, found on 12b.1's hand check

Google's own built-in calendars (holidays, birthdays) live under `group.v.calendar.google.com`; free/busy answers `notFound` for them, which made a person's whole calendar unreadable (F-412). `is-google-virtual-calendar.ts` keeps them out of the pick and out of every free/busy ask; with only such a calendar ticked, the busy read asks Google nothing (F-414).

booking by hand

`POST /schedule/bookings` calls `bookTime` with source `manual` and the owner's login: the same path as a website booking (a lead in the first stage, the timeline, the Google event, the confirmation and texts as the business has set them), any time nobody it needs is busy, inside the hours or not, never an earlier day. The time travels as the business's own date and minute of the day; the API turns it into a moment in its zone, and an hour skipped in spring is a 400 on `startMinute`. A person whose Google needs reconnecting is told so, never "try again" (F-419). The spec was amended to what 12f settled: a name is enough (F-417).

time off (`save-time-off.ts`, `find-bookings-in-the-way.ts`, `remove-time-off.ts`)

`POST /schedule/time-off` and `DELETE /schedule/time-off/:id`. Decision 3, all or nothing, needed the cancel and the move to run inside a caller's transaction, so both were split without changing behaviour: `cancel-booking-in-transaction.ts` under `cancel-booking.ts`, and `prepare-booking-move.ts` plus `apply-booking-move.ts` under `move-booking.ts`, with `every-choice-taken-error.ts` shared. `findBookingChoices` now takes a moving booking for the owner too (its own time never in its way, its Google event crossed off the main calendar only).

The save: time off already in the span is refused first, in its own words (F-422); every booking in the way must carry exactly one choice, cancel or move, and none may have begun; each move is checked as the owner books (to someone who does the service, never into the time off itself, never into the past). For a place going off, a move keeps out of that place over the span, so the same start in another room is a move (F-421). Then one transaction: every chosen booking locked in one ordered statement and compared with what the owner saw (F-423), the cancels, the moves, and the time off held last, so a booking that landed in the span meanwhile is refused by the database's no-overlap rule. Every email, text and Google change is a job saved inside it, so a refused save tells nobody anything (F-424). `remove-time-off.ts` releases only time off of this business, never a booking's held time; the dashboard CORS now allows DELETE.

the page (`frontend/`)

`/calendar`, "Calendar" after Leads in the dashboard list. `schedule-screen.tsx` picks a person or place and a week and shows each day; `book-by-hand-form.tsx` books for the person shown and keeps its

notice while the week reads again in place (F-418); `time-off-form.tsx` lists the bookings in the way and takes a choice for each, shows an API error under the booking it names (F-425), and asks nothing of a booking already begun. Four typed client calls in `lib/api-client/schedule/`, three strict schemas in `packages/shared/zod-validation/`. Logic first: the final look is a later pass.

tests

`schedule-routes.test.ts` calls the real app against the local database with Google faked: 30 tests, the four Simulate cases under their names among them, each fix's test proved able to fail by undoing the fix. Dates are chosen to hold under both the old and the 2026c tz data, which keeps Alberta on UTC-6 all year (F-416). Backend 1046 and shared 188 pass; the frontend builds and lints.

carried

F-428 (a move beside the time off whose buffer reaches into it is refused as "changed meanwhile" rather than with its booking named) and F-429 (the three new schemas hand-write one calendar-date check) are open P3s.

Findings

12b/F-412 [P2] closed - A built-in Google calendar ticked on the Calendar page makes the person's whole calendar unreadable

File: backend/lib/calendar/google-calendar-provider.ts; backend/lib/calendar/find-calendar-choices.ts **Found:** 2026-10-12 by the hand check of step 12b.1 (12h's code, merged as item-12h-done) **Why it matters:** Google lists its own holiday calendar ("Holidays in Canada", an id under `group.v.calendar.google.com`) and 12h offered it as a tick box. Free/busy answers `notFound` for it, and since one unreadable ticked calendar makes the person unreadable (12h decision 2), ticking it made Frank's whole calendar unreadable: "try again" for a picked person, left out of "any available". Seen live: the Calendar page said Google could not be read, and the API logged `Google could not read a calendar's busy times (notFound)`. **Suggested fix:** Never offer Google's built-in calendars, never ask free/busy about them, and drop one ticked before from the pick. **Resolution:** Fixed 2026-10-12 on `feature/12b-calendars`: `is-google-virtual-calendar.ts` names them; `listCalendars` skips them, `findBusyBlocks` answers them empty without asking Google, and `findCalendarChoices` leaves one ticked before out of the pick, so the next save drops it. Tests: get-busy-times.test.ts ("a built-in Google calendar ticked is never asked about, and blocks nothing"), calendar-routes.test.ts ("Google's own holiday calendar is never offered, and one ticked before is no pick at all"). Seen live after: the Calendar page reads Frank's Google again with the holiday calendar still ticked in the database. Closed 2026-10-12 by independent review of step 12b.1 (91b217c..3aafcf3): re-read google-calendar-provider.ts (listCalendars skips the suffix; findBusyBlocks asks free/busy only for the rest, answers the built-in ones `busy: []` and calls nothing when only they are ticked, while askFreeBusy keeps every 12h guarantee: one call, every asked calendar must come back, "primary" filed under the account's address, an errored

calendar throws), find-calendar-choices.ts (picked filtered, so `missing` stays empty and the save refuses the id as unknown), and the two tests, which fail if either filter is removed. Both suites pass. One residual edge is its own entry, F-414.

12b/F-413 [P2] closed - A standby day shows the person's bookable hours on the Calendar page, though customers cannot book them that day

File: backend/lib/schedule/find-schedule-week.ts:197 (and the contract comment at :45) **Found:** 2026-10-12 by independent review of step 12b.1 (scope: 91b217c..3aafcf3; lenses: quality, security, performance, tests) **Why it matters:** A day's `hours` is documented as "when customers can book", and the booking window drops a person's standby dates entirely (`applyFreeTimesRules` skips `standby.has(date)` , read through `findStandbyDates`). findScheduleWeek never reads `standby_date` , so on Carlos's standby day the owner sees his full 8 to 5 as bookable while the website offers him nothing that day. It is the one input to "when can customers book" that the calendar leaves out, and it says the opposite of what the booking window does. **Suggested fix:** Read the shown person's standby dates for the week (`findStandbyDates`) and mark the day (for example `standby: true` with `hours: []` , or the hours kept and labelled "on standby, hidden from customers"); one route test with a standby date in the week. **Resolution:** Fixed 2026-10-12: findScheduleWeek reads the person's or place's standby dates for the week (findStandbyDates, as the booking window does); a standby day answers `standby: true` and no hours, and the page says "On standby: not offered to customers". Test: schedule-routes.test.ts "a standby day shows no bookable hours, as the booking window offers none", proved able to fail. Closed 2026-10-12 by independent review of step 12b.1, check of the fixes (3aafcf3..25f814c): find-schedule-week.ts reads findStandbyDates for the shown resource over the week's seven dates, the same helper and inclusive range the booking window uses for people and places (find-free-times.ts:111), and a standby day answers `standby: true` with `hours: []` while its items stay; schedule-screen.tsx says "Closed" first, then the standby line. The test fails without the repair (no `standby` field, and 8 to 5 hours). Suites, build and lint pass.

12b/F-414 [P3] closed - Only a built-in Google calendar ticked, and a lapsed connection, still makes the person unreadable

File: backend/lib/calendar/get-busy-times.ts:39 **Found:** 2026-10-12 by independent review of step 12b.1 (scope: 91b217c..3aafcf3; lenses: quality, security, performance, tests) **Why it matters:** The F-412 repair filters built-in calendars inside the Google provider, after the "nothing ticked" check. A connection whose stored pick is only a built-in calendar (ticked under 12h, never re-saved) therefore still goes to getFreshAccessToken: if the connection needs reconnecting it throws, so the person is unreadable (left out of "any available", "Google could not be read" on the Calendar page) although nothing they ticked can count, which F-408's rule says must never block them; with a near-expired key it also refreshes it for nothing. Narrow: needs the old pick plus a lapsed permission. **Suggested fix:** Apply `isGoogleVirtualCalendar` to the pick in getBusyTimes before the empty check (line 39 and 42), so an only-built-in pick is "nothing ticked"; extend the existing built-in-calendar test with a `needs_reconnect` connection. **Resolution:** Fixed 2026-10-12: getBusyTimes counts only calendars that are not Google's

built-in ones before asking for a key, so a pick of only a holiday calendar reads nothing and never makes the person unreadable. Test: get-busy-times.test.ts "only a built-in Google calendar ticked never asks for a key, though the connection needs reconnecting". Closed 2026-10-12 by independent review of step 12b.1, check of the fixes (3aafcf3..25f814c): get-busy-times.ts applies isGoogleVirtualCalendar to the stored pick before getFreshAccessToken and again to the fresh pick, so an only-built-in pick returns [] with no key asked; a mixed pick still reaches the provider, which filters as F-412 closed. The new test (expired key, `needs_reconnect`) fails without the repair, since the token call or the reconnect error happens. The other getFreshAccessToken callers (find-calendar-choices, the booking event writers) are not on the busy-time path.

12b/F-415 [P3] closed - No test places an item that spans several local days, or reads a week with a clock change

File: backend/routes/schedule-routes.test.ts; backend/lib/schedule/find-schedule-week.ts:186 **Found:** 2026-10-12 by independent review of step 12b.1 (scope: 91b217c..3aafcf3; lenses: quality, security, performance, tests) **Why it matters:** `touches` places an item on every local day from its start's date through its last moment's, which is what lets a week of time off (12b.3's "any length") or an overnight Google event show on each day. The only cross-midnight case tested (Friday's time off ending 00:00 UTC) still passes if `touches` checked the start's date alone, so breaking multi-day placement fails nothing. The fixed one-day UTC widening and the day boundaries are also never exercised in a week holding the autumn or spring change. The code reads correct today; the tests would not notice it breaking. **Suggested fix:** Add one item spanning Friday afternoon to Monday morning (expect it on Fri, Sat, Sun and Mon; and an item ending exactly at local midnight on its start day only), and run one week across a clock change (for Edmonton, the week of 2026-10-26 or 2027-03-08). **Resolution:** Fixed 2026-10-12: two tests in schedule-routes.test.ts. "an item across several local days shows on each, and only those" (Saturday to Sunday time off), proved able to fail by placing items by their start only; "a week with the spring clock change has seven days, and places items by the clock after it" (Edmonton, March 8, 2026). Writing it showed the tz data (2026c) keeps Alberta on daylight time from March 2026, so the step's tests now say UTC-6 for November. Closed 2026-10-12 by independent review of step 12b.1, check of the fixes (3aafcf3..25f814c): the weekend test expects exactly Sat and Sun, so placing by the start alone fails it; the spring test reads the week of March 2, 2026 and expects 05:00Z on the 9th on Sunday and 06:00Z off the week, which a fixed pre-change offset (UTC-7) would put on Sunday too. Residual, not a new entry: the suggested "ends exactly at local midnight, start day only" case was not added, so the `- 1` in `touches` (find-schedule-week.ts:192) is still untested. The tz-data dependence the repair brought in is F-416.

12b/F-416 [P3] closed - The Calendar week's tests now pass only on a Node whose tz data is 2026c or later

File: backend/routes/schedule-routes.test.ts:64-78 **Found:** 2026-10-12 by independent review of step 12b.1, check of the fixes (scope: 3aafcf3..25f814c) **Why it matters:** The F-415 repair moved the suite's fixed week (Nov 16, 2026) and its UTC fixtures (tuesdayTen, fridayOff, wednesdayGoogle) to UTC-6, which

holds only under tz data 2026c, where Alberta stays on daylight time after March 8, 2026. Nothing pins the runtime (no `engines`, no `.nvmrc`), and Frank works from more than one machine: on a Node built with older tz data the same instants read an hour earlier in Edmonton, so the week tests that compare times fail although the code is right. A November week is the one choice the rule change makes ambiguous; a week between March 8 and November 1, 2026 is UTC-6 under both rule sets, as is the spring test's March 8 change. Seen here on Node v26.7.0, `process.versions.tz` 2026c, where 2026-11-17T16:00Z reads 10:00 MDT. **Suggested fix:** Move the suite's fixed week to one inside March 8 to November 1, 2026 (for example the week of June 15), keeping the UTC-6 fixtures, and say in the comment that the week was picked to read the same under either tz data. **Resolution:** Fixed 2026-10-12: the test week moved to Monday Sep 14, 2026, daylight time (UTC-6) in Edmonton under any tz data; the spring change test (March 8) holds under old and new data alike. Its 9 tests pass. Closed 2026-10-12 by independent review of step 12b.1, check of the fixes (25f814c..0ed3baa): no November date is left in the file; Sep 14, 2026 is a Monday and Sep 20 a Sunday; the whole week (with the fetch window Sep 13 00:00Z to Sep 22 00:00Z) sits inside March 8 to November 1, so Edmonton is UTC-6 under old and 2026c data alike, and every fixture reads as its comment says: 16:00Z Tue 10 am, 15:00Z Wed 9 am, 19:00Z Fri to 00:00Z Sat is Fri 1 to 6 pm, closed Sep 17 a Thursday. The weekend test keeps its shape (Sat 10 am to Sun 7 pm, 01:00Z on the 21st), so placing by the start alone still fails it and Monday stays out; the spring test is untouched and its March 8 change is in both rule sets. The week being now in the past changes nothing: the hours are resolved from the week's own Monday (find-schedule-week.ts:77-79), never from today. `npm run test --workspace=backend -- routes/schedule-routes`: 9 passed, on Node v26.7.0, tz 2026c; the old data was not run here, so that half rests on the dates above.

12b/F-417 [P3] closed - Three lines of 12b's spec promise refusals that the owner's booking path never gives

File: blueprint/context/current-feature.md:24-28 (In scope, Book by hand), :74-77 (12b.2's scene), :119-123 (`POST /schedule/bookings`) **Found:** 2026-10-12 by independent review of step 12b.2 (scope: 4e7877e..9b12f37; lenses: quality, security, performance, tests) **Why it matters:** The code follows bookTime's settled owner path (12f: "The owner's own bookings skip both", the contact rule and a required address; decision 13: a walk-in already under way is taken, only an earlier day is `in_the_past`), and the route, schema comment and test agree with it. The spec still says otherwise in three places: In scope asks for "the customer's name and phone or email, the address when the service asks one"; the contract promises "`400` on a field for a missing contact or address as `bookTime` answers" (bookTime answers neither for `manual`, so that 400 can never happen); the scene says "a time already past" is refused, but 9:00 this morning booked at 3 pm is made. Summit's Estimate is `addressMode: "required"`, and the form labels its box "Address (optional)", so the by-hand check of the scene can read as a failure, and 12b.3 or `/complete` may build on the wrong reading. Frank reads the build log, not this file, so the difference has not reached him either. **Suggested fix:** Amend the three lines to what is built and settled: name only required for the owner, the address box shown when the service asks one and left optional for the owner (12f), `in_the_past` for an earlier day in the business's zone, an earlier time today taken as a walk-in (decision 13); drop the "400 on a field for a missing contact or address" clause; say it in

the step's log entry. **Resolution:** Fixed 2026-10-12: the spec's In scope, 12b.2's story and the POST contract now say what 12f and decision 13 settled: a name is enough, phone, email and address are the owner's to give or not, only an earlier day is refused; and they name 409 reconnect_needed (F-419) and 503. Closed 2026-10-12 by independent review of step 12b.2, check of the fixes (9b12f37..39d16fc): In scope (current-feature.md:29-32) now asks only a name, with phone, email and the address "if the owner has them", and "never an earlier day"; the scene (:78) says "a day already gone"; the contract (:124-127) has `in_the_past` for an earlier day, drops the missing contact or address 400 and adds 409 `reconnect_needed` and 503 `unavailable`, which is what the route answers (schedule-routes.ts:196-222) and bookTime does for `manual` (book-time.ts:235, an earlier day only). No "phone or email" or "missing contact" wording is left in the spec. One leftover, too small to record: the Tests list (:147) still says "a past time are refused", the test's own title, whose case is yesterday, so it reads true.

12b/F-418 [P3] closed - The form's "Booked." notice is never seen: the week reloads and unmounts the form at once

File: frontend/components/schedule/book-by-hand-form.tsx:89-97; frontend/components/schedule/schedule-screen.tsx:66-69,183 **Found:** 2026-10-12 by independent review of step 12b.2 (scope: 4e7877e..9b12f37; lenses: quality, security, performance, tests) **Why it matters:** On a 201 the form sets its notice and calls `onBooked`, which is ScheduleScreen's `ask`: `setResult(null)` swaps the whole week, form included, for "Loading...". The form unmounts with its notice, and the new week mounts a fresh form with none, so "Booked. The customer hears as your settings say." is dead code and the only sign of success is the booking appearing on the week (if the reload fails, there is no sign at all). The words are also wrong for a name-only booking, where the customer hears nothing. The same remount reloads the services list after every booking. **Suggested fix:** Refresh the week without dropping the shown one (keep `result` while the new answer loads, or let `ScheduleWeek` refetch in place), or lift the notice into ScheduleScreen so it survives the reload; word it so it is true without a phone or email (for example "Booked for Mrs Lee, Thu 4:30 pm."). **Resolution:** Fixed 2026-10-12: a booking reads the week again in place (the screen's own Try again count), so the form stays, keeps its notice and does not reload the services. With no phone or email the notice says the customer hears nothing from us. Checked by hand on Diego's Friday: the notice stays and the booking shows. Closed 2026-10-12 by independent review of step 12b.2, check of the fixes (9b12f37..39d16fc): `onBooked` is now `onRefresh`, which only bumps `attempt` (schedule-screen.tsx:104,189) and never calls `setResult(null)`, so the shown week stays while the effect on `[asked, attempt]` (:58-67) reads the same week again and replaces `result` in place; `BookByHandForm` keeps its key (`resourceId`, :186), so it is not remounted, its notice survives and its services effect (book-by-hand-form.tsx:41-60, on `[personId]`) does not run again. The notice is chosen from phone or email before they are cleared (:91-103), so a name-only booking says the customer hears nothing. Only a reload that comes back unreachable swaps the week (and the form) for the Try again notice, which says why. Frontend build and lint pass.

12b/F-419 [P3] closed - An owner booking for a person whose Google needs reconnecting is told "Try again shortly", which never works

File: backend/routes/schedule-routes.ts:192-199 **Found:** 2026-10-12 by independent review of step 12b.2 (scope: 4e7877e..9b12f37; lenses: quality, security, performance, tests) **Why it matters:** For the owner, `findBookingChoices` asks the person's Google (Google wins, decision 11), and `getFreshAccessToken` throws `CalendarReconnectNeededError` for a `needs_reconnect` connection with calendars ticked, so every by-hand booking for that person is `unavailable`. The route answers 503 "Google did not answer, so that time cannot be checked. Try again shortly." That is right for an outage, but for a lapsed connection no retry ever succeeds: the owner cannot book Carlos by hand at all until Carlos reconnects or unticks, and nothing tells him so. The Calendar page shows "Google could not be read" for that week too, with the same no-reason wording. **Suggested fix:** Keep the rule, change the words to cover both cases ("Google could not be read for Carlos, so that time cannot be checked. If it keeps happening, Carlos reconnects Google on his Calendar settings."), or carry a `reconnect` reason from getBusyTimes through findBookingChoices so the route can say which. **Resolution:** Fixed 2026-10-12: when bookTime answers unavailable and the person's connection needs reconnecting, the route answers 409 reconnect_needed, "This person's Google calendar needs reconnecting... They reconnect on Settings, Calendar."; 503 stays for Google not answering. Test: "a person whose Google needs reconnecting is told so, never to try again", proved able to fail. Closed 2026-10-12 by independent review of step 12b.2, check of the fixes (9b12f37..39d16fc): for the owner a person is always named, so bookTime's `unavailable` comes only from that person's own Google read (find-booking-choices.ts:75-88; rooms are read from commitments, never Google), and `needsReconnect` reads that same person's one connection, scoped to the business (schedule-routes.ts:46-60; `calendar_connection` is unique per resource). A lapsed connection with nothing ticked never reaches Google, so it is never `unavailable` and never mislabelled; a token Google stops accepting mid-call is read after the failure, so it too gets the reconnect words. `reconnect_needed` is an existing refusal code, the form shows its message through `readSaveRefusal`, and the page it names exists (`/settings/calendar` in the build). The new test sets Carlos's seeded connection to `needs_reconnect`, expects 409 `reconnect_needed`, restores it in `finally`, and passes alone (`-t "reconnecting"`) and in the suite.

12b/F-420 [P3] closed - The "busy time" refusal test passes only after the test before it has booked

File: backend/routes/schedule-routes.test.ts:582-586 **Found:** 2026-10-12 by independent review of step 12b.2 (scope: 4e7877e..9b12f37; lenses: quality, security, performance, tests) **Why it matters:** "a busy time, time off and a past time are refused" expects `time_taken` at 6:30 pm because "Carlos is busy with Mrs Lee", a booking made by the previous test. Run alone (`-t "busy time"`, or `.only` while debugging), the first booking is made with 201 and the test fails although the rule is fine; reordered, its leftover booking is not in `made`, so its jobs are not cleaned up. The rule itself is tested; only the test's independence is not. **Suggested fix:** Make the busy booking inside the test (book once, push its id to `made`, expect 201, then book the same time again and expect 409 `time_taken`), or move the "same

time again" check into the first test. **Resolution:** Fixed 2026-10-12: the busy-time test makes its own booking first, at a time of its own; each booking test passes run alone (checked one by one with -t) and together. Closed 2026-10-12 by independent review of step 12b.2, check of the fixes (9b12f37..39d16fc): the test now books Ms Kay at 9:30 pm (1290), expects 201 and pushes her id to `made`, then books the same minute for Mr Park and expects 409 `time_taken` (schedule-routes.test.ts:589-596), so it needs nothing from the test before it and its booking's jobs are cleaned in `afterAll`. 9:30 pm is used by no other test, and the reconnect test's 11 pm still books clear of it. `-t "busy time"` alone: 1 passed, 13 skipped; the whole file: 14 passed.

12b/F-421 [P2] closed - A place's time off, with a booking moved to a time inside it, is refused as "changed meanwhile" although another room is free

File: backend/lib/schedule/save-time-off.ts:87-94,126-132; backend/lib/booking/find-booking-choices.ts:110-133 **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** The time off may be for a place (the route takes any resource), but the `move_into_time_off` check covers only the same person. For a move, findBookingChoices lists the room going off as free (its time off is not held yet, and the moving booking's own time is left out), rooms ordered by name, and holdFirstFreeChoice holds the first. When that is the room going off, the move holds it inside the span, the time off's own hold then collides, and the whole save rolls back as 409 `time_taken`, "Something in that time changed meanwhile". Every retry gives the same answer, although other rooms are free at that time. Keeping the same start in another room cannot be done either: the same start with the same person is `unchanged`, answered "That new time is not free". At Riverbend (five rooms), closing one room for an afternoon can only be saved by moving its bookings out of the afternoon or cancelling them. Found by reading the code. No test has a place's time off. **Suggested fix:** When the time off is a place, keep that place out of the move's rooms where the new span overlaps the time off (for example drop those choices from the plan, or pass the span into findBookingChoices), and count the same start in another room as a move. Add one route test with a clinic-dev room off over a booking moved inside the span. If moves for a place are meant to leave the span, say so in the spec and answer `move_into_time_off` for it. **Resolution:** Fixed 2026-10-12: a place's time off passes the place to prepareBookingMove as `leavingPlace`; the move's choices drop that place wherever the new span overlaps the time off, and a booking still in that place is never "already there" (prepare and applyBookingMove both), so the same start in another room is a move. With no other room free the move answers `time_taken`, "That new time is not free". Route test "a room going off" (Room A off 9 am to 1 pm, one booking kept at 10 in Room B, one moved to noon in Room B); it fails with either half undone. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (28fe13a..107e315): the customer's and the owner's single move are unchanged (no leavingPlace, so both "already there" checks reduce to their old form); the filter uses the span with buffers against the time off, half-open, and drops only the room going off; the test covers the same start and a new time inside the span.

12b/F-422 [P3] closed - Time off over time off already there is refused as "changed meanwhile", every time

File: backend/lib/schedule/save-time-off.ts:126-132; backend/routes/schedule-routes.ts:393-399 **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** findBookingsInTheWay looks only at bookings, so time off already in the span is first met by the final holdTime. Its refusal answers 409 `time_taken`, "Something in that time changed meanwhile. Nothing was saved: look again." Marking Carlos off all Friday when Friday afternoon is already off (12b.1's own scene) always gets that answer, and looking again changes nothing. The owner has to guess that the afternoon must be removed first, after choosing a move or cancel for every booking in the way (each one correctly undone). **Suggested fix:** Before the transaction, read the resource's active time off over the span and refuse it in its own words ("Already off from 1 pm to 5 pm that day: remove it first, or start after it"). Keep the in-transaction refusal for a real "meanwhile". Add one route test. **Resolution:** Fixed 2026-10-12: saveTimeOff reads the resource's active time off over the span before the bookings in the way, and refuses it as 409 `already_off`, "Already off from <its start> to <its end>. Remove that time off first, or pick a time around it.", before any booking is chosen for. The in-transaction refusal stays for a real meanwhile. Route test "time off over time off already there"; it fails with the check undone (answers bookings_in_the_way). Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (28fe13a..107e315): findCommitments' half-open overlap means time off that only touches the span is not refused; a test covers it.

12b/F-423 [P3] closed - A booking its customer moves while the save runs is still cancelled or moved by it

File: backend/lib/schedule/save-time-off.ts:66-80,117-125; backend/lib/booking/cancel-booking-in-transaction.ts:57-61; backend/lib/booking/apply-booking-move.ts:206-215 **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** The bookings in the way are read before the transaction, and before each move's Google read, which can take seconds. Once a row is locked, the cancel checks only that it is confirmed and has not begun, and applyBookingMove only that it is not cancelled, not begun and not already at the new start. Suppose a customer moves Friday's booking to next Wednesday through their link in that window. The owner's Friday time off then cancels the Wednesday booking (the customer is told of a move, then of a cancel), or moves it again from Wednesday, although it is no longer in the way. Nothing is left half done, but the save acts on a booking that changed meanwhile instead of refusing. The window is narrow. **Suggested fix:** In the time off save, after each row is locked, compare its start and person with the entry read before the transaction, and throw ChangedMeanwhileError when they differ. One way is an optional `expected: { startsAt, personId }` on cancelBookingInTransaction and applyBookingMove, used only by this save. **Resolution:** Fixed 2026-10-12: inside the transaction, before any cancel or move, every booking chosen for is locked (one statement, ordered by id) and compared with what the owner saw: still confirmed, same start, same person, or ChangedMeanwhileError undoes the lot (409 `time_taken`). Done in the save rather than as an `expected` option on the shared cancel and move, so the customer's paths

are untouched. Route test "a booking its customer moves while the save runs" (Ben moved a week on during Ana's Google read): Ben keeps his new time, no job added; it fails with the check undone. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (28fe13a..107e315): one SELECT ... ORDER BY id FOR UPDATE first in the transaction, so two saves queue rather than deadlock; the later locks in cancel and move are no-ops in the same transaction; the test refuses from inside the transaction.

12b/F-424 [P2] closed - The Simulate case "a move to a time taken meanwhile saves nothing, not even the cancel" never reaches the transaction

File: backend/routes/schedule-routes.test.ts:802-828 (and :830-874) **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** Cara's 9 am booking is made before the save, so prepareBookingMove refuses Ana's move (the `bookingId: ana` the test asserts) before the transaction starts, and Ben's cancel is never tried. The test would still pass if the cancels ran in their own transaction ahead of the moves, which is exactly what decision 3 forbids. The one test that fails inside the transaction ("two moves to one free time") has no cancel, and no test checks that a refused save leaves no job (the cancellation email, the worker text, the Google removal). So "no customer told of a cancel that did not finish" is untested. Two smaller rules are untested too: a booking's held time is never removable by DELETE (404, the booking still held), and an owner move to a time already gone is 400 `in_the_past`. **Suggested fix:** Make this case fail inside the transaction after a cancel: cancel Ben, and move Ana and a third booking to the same free time, so the second hold fails. Then assert Ben is `confirmed`, `emailsOf(ben)` is empty, Ana is unmoved and there is no time off. Also add a DELETE of a booking's commitment id (expect 404) and a move to yesterday (expect 400 `in_the_past`). **Resolution:** Fixed 2026-10-12: the old case is renamed "a move to a time already taken is refused before anything is saved" (what it proves). The new "a move to a time taken meanwhile saves nothing, not even the cancel" cancels Ben, then moves Ana and Cara to the same free time, so the second hold fails inside the transaction after the cancel: Ben confirmed with no job added (emails, texts, Google), Ana and Cara unmoved, no time off. It fails when the cancels run outside the transaction. Added: a booking's own held time DELETEd is 404 and stays active; a move to yesterday is 400 `in_the_past`. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (28fe13a..107e315): each move is free alone, so the second hold fails inside the transaction after the cancel; the job count is meaningful (the cancel's email, Google removal and worker text all carry bookingId); the DELETE 404 and in_the_past tests are present.

12b/F-425 [P3] closed - The time off form shows nothing when the API refuses a move's day or time

File: frontend/components/schedule/time-off-form.tsx:125-128 (the shown keys at :160-201 and :287) **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** The route answers `field: "bookings.N.startMinute"` for a move into the hour skipped in spring (schedule-routes.ts:299). The schema answers `bookings.N.date` ("Pick a day.") when the owner clears a move's day box, and can also answer `resourceId` or `bookings`. The form files the error under that key but shows only from., *to.*, `to` and

`booking-<id>`, so the button comes back and nothing is shown. Also, a booking already begun gets "Choose move or cancel." from the form's own check, although the form offers it neither. **Suggested fix:** Map `bookings.N.` *to* `booking-<id>` *of the N-th booking sent (the order matches the list), and show any other field in the notice. Leave the begun ones out of the choice check, since their own notice already says to start the time off after them.* ***Resolution:*** *Fixed 2026-10-12: the form maps* `bookings.N.` to the N-th booking sent and shows it under that booking, shows its own span fields under their boxes, and puts any other field in the notice. Begun bookings are left out of the choice check and not sent; the API's own "already begun" answer is shown. Checked in the browser at http://localhost:3400/calendar?date=2026-10-15: Carlos off Thu Oct 15, Mrs Lee (test) moved with the day cleared shows "Pick a day." under her booking; nothing saved. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (28fe13a..107e315): the route's bookings.N index is the array the form sends, begun bookings skipped before sending, so N maps to the right booking.

12b/F-426 [P3] closed - The time off contract in the spec differs from what the route answers

File: blueprint/context/current-feature.md:129-138; backend/routes/schedule-routes.ts:348-399 **Found:** 2026-10-12 by independent review of step 12b.3 (scope: 2503027..5aed73b; lenses: quality, security, performance, tests) **Why it matters:** The contract lists only 409s, and `bookings_in_the_way` always with `{ bookings }`. The route also answers 400 `in_the_past` and 503 `unavailable` for a move, and 400 for a skipped clock time. It answers `bookings_in_the_way` with a `bookingId` and no `bookings` for a move whose booking changed, and for a person who does not do the service. For that person the words "A booking changed meanwhile" are wrong. The form copes, since it branches on `bookings`, but `/complete` and the build log build on the spec. **Suggested fix:** Amend the contract lines to the answers the route gives. Give the person who does not do the service its own words (for example `not_found`, "That person does not do this service"). **Resolution:** Fixed 2026-10-12: the POST /schedule/time-off contract in current-feature.md now lists every answer the route gives: 404, 409 already_off, 409 bookings_in_the_way with and without bookings, per move 400 bad_request, 400 in_the_past, 409 time_taken, 503 unavailable, 400 { field } including bookings.N.startMinute, and 409 time_taken for a meanwhile. A person who does not do the service now has its own words, 400 bad_request "That person does not do this service. Pick someone who does." with bookingId, checked against the booking's own people before the move is prepared. Route test covers it; it fails with the check undone. Not closed by the check of the fixes (28fe13a..107e315): the contract missed 409 no_business_hours (schedule-routes.ts:272). Fixed again 2026-10-12: the clause is added. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (107e315..dda246d): the contract (current-feature.md:133-145) now names every answer of the POST /time-off handler, 409 no_business_hours included (schedule-routes.ts:272); the move reasons not_found, already_cancelled and already_started land in the bookings_in_the_way fallback, as written.

12b/F-427 [P3] closed - The two "meanwhile" tests do not prove the refusal came from inside the transaction

File: backend/routes/schedule-routes.test.ts:861-862, :896-897 **Found:** 2026-10-12 by independent review of step 12b.3, check of the fixes (scope: 28fe13a..107e315; lenses: tests) **Why it matters:** Both tests check only `error.code` `time_taken`. A move refused before the transaction answers the same code, with a `bookingId`. If a later change made either move refuse early, both would still pass without reaching the transaction, the gap F-424 was about. **Suggested fix:** Add `expect(body).not.toHaveProperty("bookingId")` to both. **Resolution:** Fixed 2026-10-12: both tests now assert no bookingId, with a comment saying why. Proved able to fail: the route's meanwhile answer given a bookingId made both tests fail; put back after. Closed 2026-10-12 by independent review of step 12b.3, check of the fixes (107e315..dda246d): both tests assert no bookingId (schedule-routes.test.ts:862-865, :905-908); every move refused before the transaction answers with one, and the only time_taken without one is the in-transaction catch (save-time-off.ts:194).

Independent review

The passing receipt over the whole feature, `91b217c..b2d83d4`. Each step had its own fresh reviewer and checks of its fixes before it (F-412 to F-427, recorded in Findings above).

Status: passed **Target commit:** b2d83d41010b164bc17b10634f5fce348cd06fc2 **Base commit:** 91b217c3a327f1341d44098e0fd0817c4600895e **Base ref:** main **Spec hash:** 95ee5a6503b07fe890210b60dce3fa9a8566e7efac2897951b73a8529507b138 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-12T23:26:18Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-12T23:33:00Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete `91b217c3a327f1341d44098e0fd0817c4600895e..b2d83d41010b164bc17b10634f5fce348cd06fc2` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run test --workspace=@scheduleads-app/shared`: pass (26 files, 188 tests)
- `npm run test --workspace=backend`: pass (89 files, 1046 tests, against the local seeded `scheduleads_dev`)
- `npm run build --workspace=frontend`: pass (`/calendar` built; route types and booking component built by its hooks)
- `npm run lint --workspace=frontend`: pass

Evidence

- Freshness confirmed before review: `HEAD` = target, `git merge-base main HEAD` = base, SHA-256 of `current-feature.md` = spec hash, branch `feature/12b-calendars`; working tree differs from the target only in `review.md` and the owner's untracked `blueprint/ai-voice-proposal.md` (not code, left alone).
- Whole delta read (43 files): `backend/routes/schedule-routes.ts`, `backend/lib/schedule/*`, the extracted `cancel-booking-in-transaction.ts` / `prepare-booking-move.ts` / `apply-booking-move.ts` (the cancel and move extractions are line-for-line moves of the old bodies), `find-booking-choices.ts`, the Google virtual-calendar changes, `app.ts` mounting under dashboard CORS, CSRF and no-store, the three Zod schemas, the four frontend client calls, the three schedule components and the page.
- Security: every query filters on the session's business first; reads gate on `organization: ["update"]` or the caller's own linked person (403 otherwise), writes on `requirePermissionMiddleware`; a choice can only name a booking already found in that business's span, and a move's person must be one of that booking's offered people; DELETE is non-simple, so CORS preflight guards it; customer names render as React text.
- Decision 3 (all or nothing): the chosen bookings are locked in one ordered `FOR UPDATE` and compared with what the owner saw, then cancels, moves and the time off hold run in one transaction; the database's no-overlap rule refuses a booking that landed in the span meanwhile; jobs (emails, texts, Google) are enqueued inside the transaction; route tests prove a refused save leaves no job.
- Tests: all four Simulate cases are saved under their names, plus removing time off, a room going off, a booking's own held time not removable, a member refused, and DST and multi-day placement on the week; no skipped or focused tests in the new suites.

Findings

- F-428 [P3] open: a move placed right beside the time off whose buffer reaches into it is refused as "changed meanwhile" with no `bookingId`, every time (the same-person guard ignores buffers).
- F-429 [P3] open: the three new schedule schemas hand-write one calendar-date check, where the package already uses `z.iso.date()`.
- No P0 or P1 found; no P0 or P1 in the ledger is `open` or `fixed`.

Remaining risk

- No browser walk of the Calendar page in this review (Check not required); the forms were read, not driven.
- A time off save with many moves prepares each one in turn, each with its own Google read, before the transaction: unverified, not measured, bounded at 100 choices.
- The untracked `blueprint/ai-voice-proposal.md` is present in the working tree; it is the owner's note, outside the review scope, but it is a path that differs from the target.

- No verification command was unavailable.